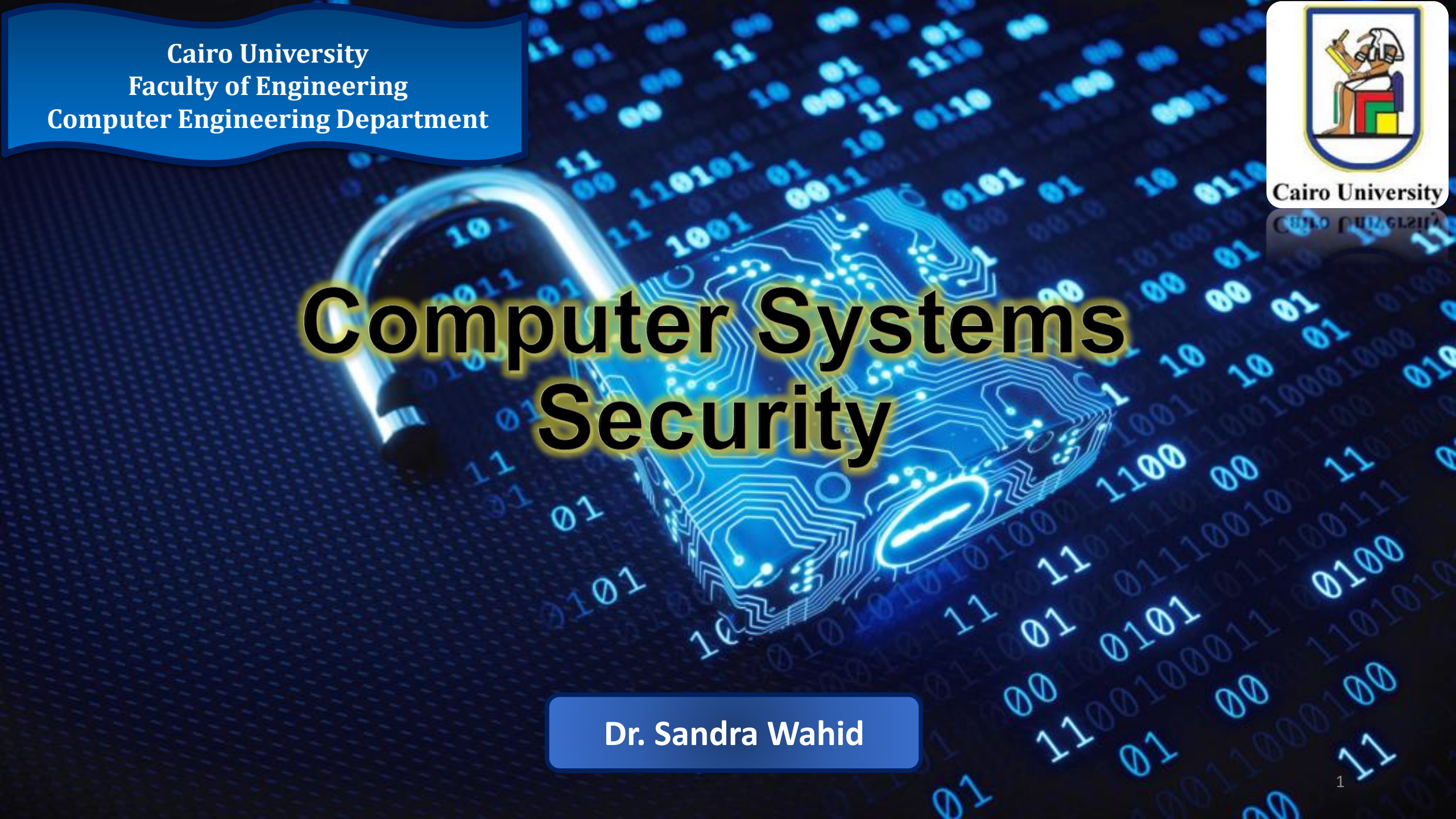
Cairo University
Faculty of Engineering
Computer Engineering Department

Cairo University



Computer Systems Security

Dr. Sandra Wahid

Diffie-Hellman Key Exchange

- The purpose of the algorithm is to enable two users to **securely exchange a key** that can then be used for subsequent symmetric encryption of messages.
- Depends for its effectiveness on the difficulty of computing **discrete logarithms**.

Involved Mathematical Concepts:

- A **primitive root of a prime number** *p* is one whose powers modulo *p* generate all the integers from *1* to *p - 1*.
  - If *a* is a primitive root of the prime number *p*, then the numbers $a \bmod p, a^2 \bmod p, \ldots, a^{p-1} \bmod p$ are distinct and consist of the integers from *1* through *p - 1* in some permutation.
- For any integer *b* and a primitive root *a* of prime number *p*, we can find a unique exponent *i* such that: $b \equiv a^i \pmod{p} \quad \text{where } 0 \leq i \leq (p-1)$
  - The exponent *i* is referred to as the discrete logarithm of *b* for the base *a*, *mod p*.
  - Expressed as ***dloga,p(b)***.

Diffie-Hellman Key Exchange

- There are **two publicly known** numbers:
  - a prime number ***q***
  - an integer ***a*** that is a primitive root of *q* and *a* < *q*
- Suppose the users A and B wish to create a shared key.
- Each user (eg. A) generates their key
  - chooses a **private key**: $x_A < q$
  - computes their **public key**: $y_A = a^{x_A} \bmod q$
- User A computes the key as $K = y_B^{x_A} \bmod q$
- Similarly, user B computes the key as $K = y_A^{x_B} \bmod q$

These two calculations produce identical results, ***How??***

$$
\begin{aligned}
K &= (Y_B)^{X_A} \bmod q \\
&= (\alpha^{X_B} \bmod q)^{X_A} \bmod q \\
&= (\alpha^{X_B})^{X_A} \bmod q \qquad \text{by the rules of modular arithmetic} \\
&= \alpha^{X_B X_A} \bmod q \\
&= (\alpha^{X_A})^{X_B} \bmod q \\
&= (\alpha^{X_A} \bmod q)^{X_B} \bmod q \\
&= (Y_A)^{X_B} \bmod q
\end{aligned}
$$

K is a shared symmetric secret key between A and B.

Diffie-Hellman Key Exchange

Diffie-Hellman Security:

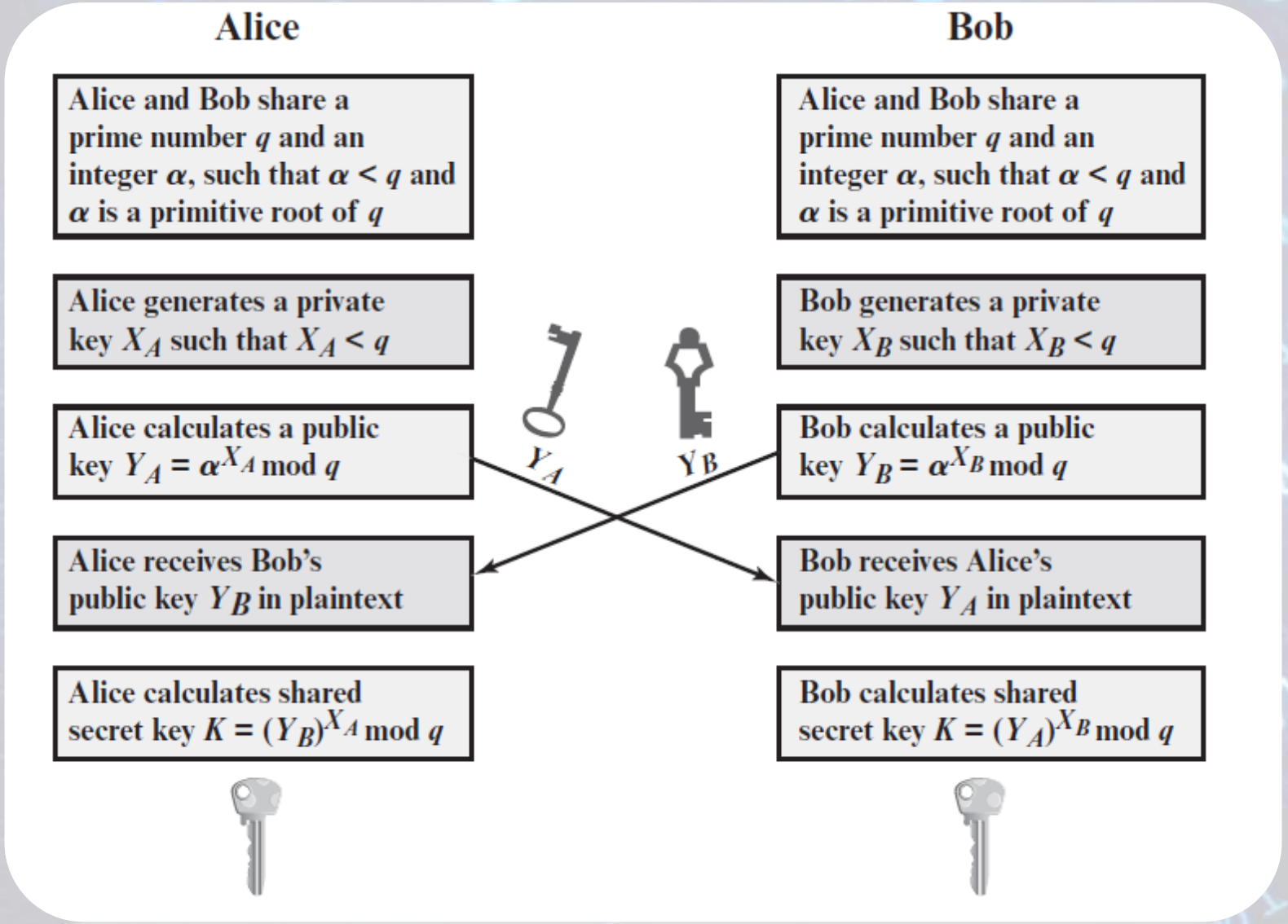
- Because x_A and x_B are private, an adversary only has: q, a, y_A, and y_B.
- Thus, the adversary is forced to solve the discrete logarithm problem, which is **hard**.
- For example, to determine the private key of user B, an adversary must compute:

  $X_B = \text{dlog}_{\alpha,q}(Y_B)$ then calculate K as: $y_A^{x_B} \bmod q$
- For large primes, calculating discrete logarithms is considered infeasible.

Example:

- Users Alice & Bob wish to share a secret key:
- Agree on prime $q=353$ and $a=3$
- Select random private keys:
  - A chooses $x_A=97 < 353$, B chooses $x_B=233 < 353$
- Compute respective public keys:
  - $y_A=3^{97} \bmod 353 = 40$ (Alice)
  - $y_B=3^{233} \bmod 353 = 248$ (Bob)
- Compute shared secret key as:
  - $K= y_B^{x_A} \bmod 353 = 248^{97} \bmod 353 = 160$ (Alice)
  - $K= y_A^{x_B} \bmod 353 = 40^{233} \bmod 353 = 160$ (Bob)

Diffie-Hellman Key Exchange



Man-in-the-Middle Attack

- The protocol described is insecure against a man-in-the-middle attack.
- Suppose Alice and Bob wish to exchange keys, and Darth is the adversary. The attack proceeds as follows:

1. Darth prepares for the attack by generating two random private keys X_{D1} and X_{D2} and then computing the corresponding public keys Y_{D1} and Y_{D2}.
2. Alice transmits Y_A to Bob.
3. Darth intercepts Y_A and transmits Y_{D1} to Bob. Darth also calculates $K2 = (Y_A)^{X_{D2}} \bmod q$.
4. Bob receives Y_{D1} and calculates $K1 = (Y_{D1})^{X_B} \bmod q$.
5. Bob transmits Y_B to Alice.
6. Darth intercepts Y_B and transmits Y_{D2} to Alice. Darth calculates $K1 = (Y_B)^{X_{D1}} \bmod q$.
7. Alice receives Y_{D2} and calculates $K2 = (Y_{D2})^{X_A} \bmod q$.

At this point, Bob and Alice think that they share a secret key, but instead Bob and Darth share secret key $K1$ and Alice and Darth share secret key $K2$. All future communication between Bob and Alice is compromised in the following way.

1. Alice sends an encrypted message M: $E(K2, M)$.
2. Darth intercepts the encrypted message and decrypts it to recover M.
3. Darth sends Bob $E(K1, M)$ or $E(K1, M')$, where M' is any message. In the first case, Darth simply wants to eavesdrop on the communication without altering it. In the second case, Darth wants to modify the message going to Bob.

ElGamal Cryptographic System

- A public-key scheme based on discrete logarithms, closely related to the Diffie-Hellman technique.

As with Diffie–Hellman, the global elements of Elgamal are a prime number q and α, which is a primitive root of q. User A generates a private/public key pair as follows:

1. Generate a random integer X_A, such that $1 < X_A < q - 1$.
2. Compute $Y_A = \alpha^{X_A} \bmod q$.
3. A's private key is X_A and A's public key is $\{q, \alpha, Y_A\}$.

Any user B that has access to A's public key can encrypt a message as follows:

1. Represent the message as an integer M in the range $0 \leq M \leq q - 1$. Longer messages are sent as a sequence of blocks, with each block being an integer less than q.
2. Choose a random integer k such that $1 \leq k \leq q - 1$.
3. Compute a one-time key $K = (Y_A)^k \bmod q$.
4. Encrypt M as the pair of integers (C_1, C_2) where

$$C_1 = \alpha^k \bmod q; \; C_2 = KM \bmod q$$

User A recovers the plaintext as follows:

1. Recover the key by computing $K = (C_1)^{X_A} \bmod q$.
2. Compute $M = (C_2 K^{-1}) \bmod q$.

The security of Elgamal is based on the difficulty of computing **discrete logarithms**.

- To recover A's private key, an adversary would have to compute:

$$X_A = \text{dlog}_{\alpha,q}(Y_A)$$

- To recover the one-time key K, an adversary would have to determine the random number k:

$$k = \text{dlog}_{\alpha,q}(C_1)$$

ElGamal Cryptographic System

Demonstrating why the scheme works:

$K = (Y_A)^k \bmod q$	K is defined during the encryption process
$K = (\alpha^{X_A} \bmod q)^k \bmod q$	substitute using $Y_A = \alpha^{X_A} \bmod q$
$K = \alpha^{kX_A} \bmod q$	by the rules of modular arithmetic
$K = (C_1)^{X_A} \bmod q$	substitute using $C_1 = \alpha^k \bmod q$

Next, using K, we recover the plaintext as

$$C_2 = KM \bmod q$$
$$(C_2K^{-1}) \bmod q = KMK^{-1} \bmod q = M \bmod q = M$$

Example:

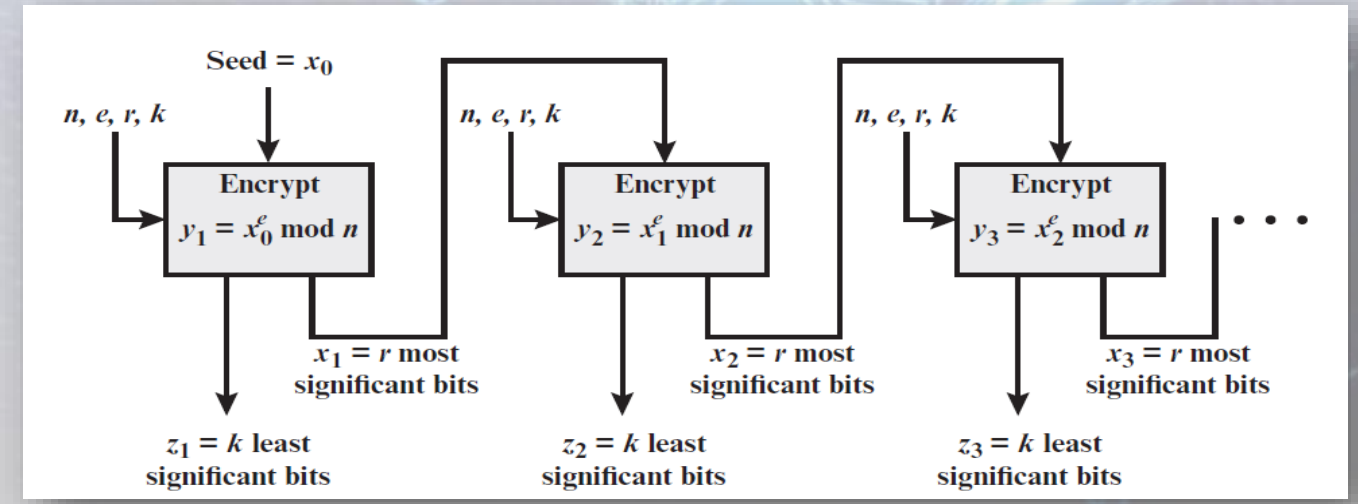
- $q=19$ and $a=10$
- Alice computes her key:
  - A chooses $x_A=5$ & computes $y_A=10^5 \bmod 19 = 3$
- Bob sends message $M=17$ as $(11,5)$ by
  - choosing random $k=6$
  - computing $K = y_{A_k}^{k} \bmod q = 3^6 \bmod 19 = 7$
  - computing $C_1 = a^k \bmod q = 10^6 \bmod 19 = 11$
    $C_2 = KM \bmod q = 7.17 \bmod 19 = 5$
- Alice recovers original message by computing:
  - recover $K = C_1^{xA} \bmod q = 11^5 \bmod 19 = 7$
  - compute inverse K^{-1} using the Extended Euclidean Algorithm $= 7^{-1} \bmod 19 = 11$
  - recover $M = C_2 \ K^{-1} \bmod q = 5.11 \bmod 19 = 17$

a=19, b=7 → gcd(19,7) is guaranteed to be 1 since q=19 is a prime number so inverse is *y* after appling Ext. Eclud.

Pseudorandom Number Generation Based on an Asymmetric Cipher

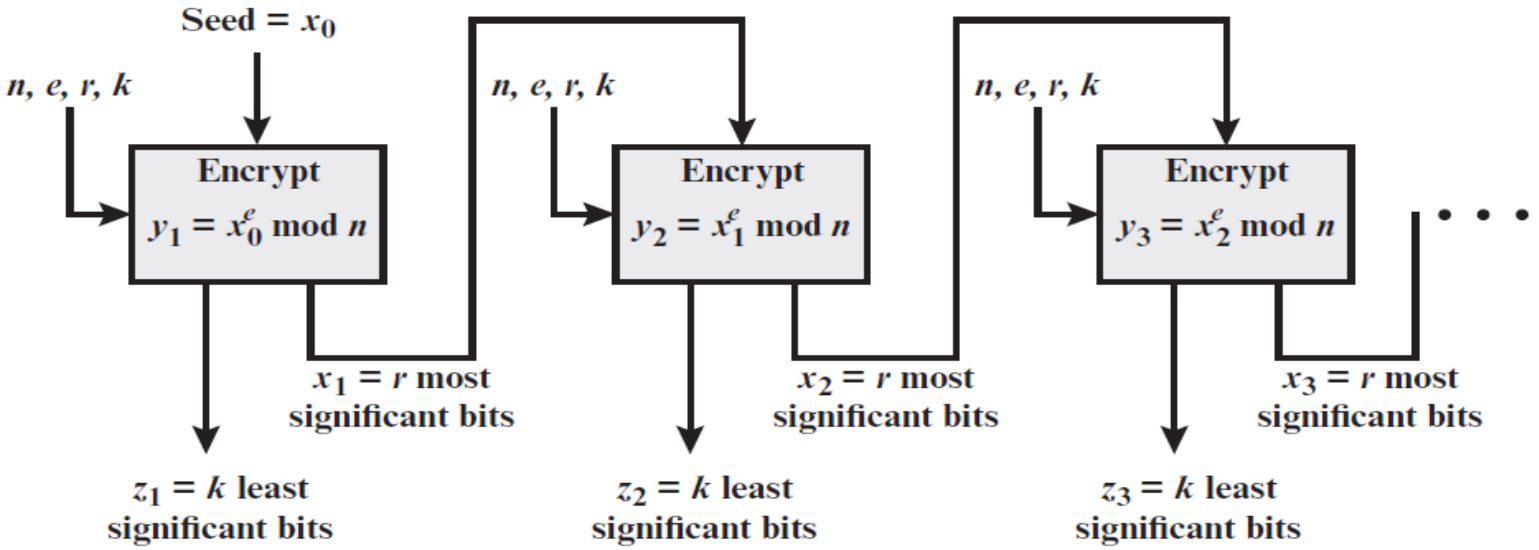
PRNG based on RSA:

- Known as the Micali–Schnorr PRNG.
- This PRNG has much the same structure as the **output feedback (OFB)** mode used as a PRNG.
- The encryption algorithm is RSA rather than a symmetric block cipher.
- A portion of the output is fed back to the next iteration of the encryption algorithm and the remainder of the output is used as pseudorandom bits.
  - The motivation for this separation of the output into two distinct parts is so that the pseudorandom bits from one stage do not provide input to the next stage.



Pseudorandom Number Generation Based on an Asymmetric Cipher

PRNG based on RSA:



Setup Select p, q primes; $n = pq$; $\phi(n) = (p - 1)(q - 1)$. Select e such that $\gcd(e, \phi(n)) = 1$. These are the standard RSA setup selections
In addition, let $N = [\log_2 n] + 1$ (the bitlength of n).
Select r, k such that $r + k = N$.

Seed Select a random seed x_0 of bitlength r.

Generate Generate a pseudorandom sequence of length $k \times m$ using the loop
for i **from** 1 to m **do**

$$y_i = x_{i-1}^e \bmod n$$
$$x_i = r \text{ most significant bits of } y_i$$
$$z_i = k \text{ least significant bits of } y_i$$

Output The output sequence is $z_1 \| z_2 \| \ldots \| z_m$.


Thank You